

平成 27 年度

筑波大学大学院博士課程
システム情報工学研究科
コンピュータサイエンス専攻

博士前期課程（一般入学試験 2 月期）

試験問題 基礎科目（情報基礎）

Fundamentals of Computer Science

[注意事項] [Instructions]

1. 試験開始の合図があるまで問題の中を見てはいけません。また、筆記用具を手に持つてはいけません。
Do NOT open this booklet before the examination starts. In addition, do NOT hold any pen or pencil in your hand.
2. 試験開始の合図のあとで、全ての解答用紙の定められた欄に、研究科、専攻、受験番号を記入すること。
Fill in the designated spaces on each answer sheet with the name of the graduate school, name of your main (major) field, and the examination number after the examination starts.
3. この問題は全部で 6 ページ（表紙を除く）です。1～3 ページは日本語版、4～6 ページは英語版です。
This booklet consists of 6 pages, excluding the cover sheet. The Japanese version is shown on pages 1-3 and the English version on pages 4-6.
4. 解答用紙（罫線有り）を 2 枚、下書き用紙（白紙）を 1 枚配布します。
You are given two ruled answer sheets and one white draft sheet.
5. 問題は全部で 2 問あります。問題 I の解答を 1 枚目、問題 II の解答を 2 枚目の解答用紙に、必ず分けて記入すること。
There are two problems. Write the answers to Problem I on the first answer sheet and the answers to Problem II on the second answer sheet.
6. 解答用紙に解答を記述する際に、問題 I の解答か問題 II の解答かを必ず明記すること。
When writing the answers, clearly label the problem number on each answer sheet.

平成 27 年 2 月 2 日

問題 I の解答は 1 枚目の解答用紙に記入すること。問題 I の解答であることを明記すること。

問題 I 有向グラフを隣接行列 $A = (a_{i,j})$ で表現する。行列 A の要素は 0 または 1 とする。すなわち、頂点 i から頂点 j への辺がひとつ有るとき $a_{i,j} = 1$ 、無いとき $a_{i,j} = 0$ とする。対角要素 $a_{i,i}$ はすべて 0 とする。有向グラフの頂点数が N のとき、行列 A は N 次正方行列となる。以下、要素の添字は 0 から始めるものとする。

図 1 は、有向グラフの頂点を、ある条件を満たすようにソートするための C 言語のプログラムの一部である。ここで隣接行列は、図 1 の 2 行目に示したように 2 次元配列を用いて記述する。ただし、定数 N は頂点数を表すとする。

以下の設問に答えなさい。

(1) 式 (1) に示した隣接行列 A で表された有向グラフを図で描きなさい。

$$A = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix} \quad (1)$$

- (2) 関数 `count_indegree()` は、すべての頂点の入次数（各頂点に向う辺の数）を求める関数である。頂点 i の入次数は `indegree[i]` に格納される。図 1 の (A) の部分を埋めて関数 `count_indegree()` を完成させなさい。
- (3) 2 次元配列 `a[N][N]` が式 (1) に示された隣接行列 A を表しているとする。関数 `sort()` が呼び出された後の配列 `sorted[]` の要素をすべて書き出さなさい。
- (4) 関数 `sort()` の戻り値が定数 N と等しくなるのは、有向グラフがどのような性質を備えるときか答えなさい。
- (5) 上記の性質を備える有向グラフが与えられたとき、関数 `sort()` は、頂点をある条件を満たすようにソートする関数である。その条件とはどのようなものか答えなさい。

```

#define N 5
char a[N][N];
int sorted[N], indegree[N], nodestack[N], stackp = 0;

void count_indegree() {
    int tail, head;
    for (head = 0; head < N; head++) {
        indegree[head] = 0;



    }
}

void remove_node(int node) {
    int head;
    for (head = 0; head < N; head++) {
        if (a[node][head] == 1) {
            indegree[head]--;
            if (indegree[head] == 0) {
                nodestack[stackp] = head;
                stackp++;
            }
        }
    }
}

int sort(int sorted[]) {
    int count = 0, node;
    count_indegree();
    for (node = 0; node < N; node++) {
        if (indegree[node] == 0) {
            nodestack[stackp] = node;
            stackp++;
        }
    }
    while (stackp > 0) {
        stackp--;
        node = nodestack[stackp];
        sorted[count] = node;
        count++;
        remove_node(node);
    }
    return count;
}

```

図 1: 有向グラフの頂点をソートする、C 言語によるプログラムの一部

問題 II の解答は 2 枚目の解答用紙に記入すること。問題 II の解答であることを明記すること。

問題 II デジタル回路を考える。真理値表が以下のように与えられる場合、以下の設問に答えなさい。

a	b	c	d	e
0	0	0		0
0	0	1		0
0	1	0		0
0	1	1		1
1	0	0		0
1	0	1		1
1	1	0		0
1	1	1		1

(1) a 、 b 、 c を入力とし、 e を出力とする場合の論理式を求めなさい。

(2) d が以下の論理式で与えられ、 a 、 b 、 c 、 d を入力とし、 e を出力とする場合を考える。

$$d = a \cdot b + \bar{b} \cdot c$$

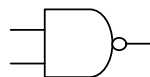
(2-1) 真理値表の d の値をすべて埋めなさい。

(2-2) 出力 e に関して以下のカルノー図を完成させなさい。ただし、 a 、 b 、 c 、 d の組合せがない場合の e は「*」と記入しなさい。

ab \ cd	00	01	11	10
00	0		*	
01				
11				
10		1		

(2-3) (2-2) のカルノー図をもとにして、 e の論理式を a 、 c 、 d を使って求めなさい。

(2-4) (2-3) の論理式をもとにして、 e を出力とする回路図を書きなさい。ただし、2 入力 NAND ゲートを 4 つまで使用できるものとする。NAND ゲートは以下の図を用いること。



Write the answers to Problem I on the first answer sheet, and clearly label it at the top of the page as “Problem I.”

Problem I An adjacency matrix $A = (a_{i,j})$ can be used to represent a directed graph. Suppose that $a_{i,j}$ is 0 or 1. If there is an edge from node i to node j , $a_{i,j}$ is 1; otherwise $a_{i,j}$ is 0. Every diagonal element $a_{i,i}$ is 0. When the number of nodes is N , the adjacency matrix A is a square matrix of order N . Here after we begin indices from 0.

Figure 1 shows a part of a C language program; this program sorts nodes of a directed graph such that the nodes satisfy a certain condition. An adjacency matrix is represented by using a two-dimensional array defined in the second line in Figure 1. Constant N expresses the number of nodes.

Answer the following questions.

- (1) Illustrate the directed graph represented by the following adjacency matrix A with a diagram.

$$A = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix} \quad (1)$$

- (2) Function `count_indegree()` calculates the indegree (the number of edges directed into a node) of each node. The indegree of node i will be stored in `indegree[i]`. Fill in the box (A) in Figure 1 to complete the function `count_indegree()`.
- (3) Suppose that two-dimensional array `a[N][N]` represents the adjacency matrix A expressed by exp. (1). Write all elements of array `sorted[]` when the processing `sort()` is finished.
- (4) When the directed graph has a certain feature, the returned value of function `sort()` becomes equal to the constant N . Describe the feature.
- (5) When a directed graph with the above feature is given, function `sort()` sorts the nodes of the directed graph such that the nodes satisfy a certain condition. Describe the condition.

```

#define N 5
char a[N][N];
int sorted[N], indegree[N], nodestack[N], stackp = 0;

void count_indegree() {
    int tail, head;
    for (head = 0; head < N; head ++) {
        indegree[head] = 0;



(A)


    }
}

void remove_node(int node) {
    int head;
    for (head = 0; head < N; head ++) {
        if (a[node][head] == 1) {
            indegree[head] --;
            if (indegree[head] == 0) {
                nodestack[stackp] = head;
                stackp ++;
            }
        }
    }
}

int sort(int sorted[]) {
    int count = 0, node;
    count_indegree();
    for (node = 0; node < N; node ++) {
        if (indegree[node] == 0) {
            nodestack[stackp] = node;
            stackp ++;
        }
    }
    while (stackp > 0) {
        stackp --;
        node = nodestack[stackp];
        sorted[count] = node;
        count ++;
        remove_node(node);
    }
    return count;
}

```

Figure 1: A part of a program in the C language; this program sorts nodes of a directed graph

Write the answers to Problem II on the second answer sheet, and clearly label it at the top of the page as “Problem II.”

Problem II Let us consider a digital circuit. For the truth table given below, answer the following questions.

a	b	c	d	e
0	0	0		0
0	0	1		0
0	1	0		0
0	1	1		1
1	0	0		0
1	0	1		1
1	1	0		0
1	1	1		1

- (1) When a , b and c are input signals and e is the output, derive a logical formula for e in terms of a , b and c .
- (2) Let us consider the case when the logical formula for d is given below, a , b , c and d are input signals and e is the output.

$$d = a \cdot b + \bar{b} \cdot c$$

- (2-1) Fill the logical patterns for d in the truth table above.
- (2-2) Complete the Karnaugh map below for e . Write the input/output pattern(s) that are not included in the table above as “*”.

cd \ ab	00	01	11	10
00	0		*	
01				
11				
10		1		

- (2-3) Using the Karnaugh map completed in (2-2), derive a logical formula for e in terms of a , c and d .
- (2-4) According to the logical formula of (2-3), derive a schematic for the logical statement for e . Use only not more than four two-input NAND gates. Here, use the symbol of the NAND gate given below.

